DBMS Quiz Evaluation Project API Contracts

This document focuses on request input, response output, validation expectations, and one example per module based on the current Node.js backend.

Quick Summary Table

Module	Main APIs	Input Focus	Output Focus

Auth	`POST /api/auth/signup`
	`POST /api/auth/login`
	`GET /api/auth/me`
		Email, password, role	User, session, profile

Users	`GET /api/users` `POST /api/users`	Email, role, optional auth mapping	User rows

Exams	`GET /api/exams`
	`GET /api/exams/:id`
	`POST /api/exams`
	`POST /api/exams/:id/questions`
		Exam metadata and question data	Exam objects and question objects

Submissions	`POST /api/submissions/start`
	`POST /api/submissions/:id/save`
	`POST /api/submissions/:id/submit`
	`GET /api/submissions/:id`
		Exam id, answer_data	Submission object with status/hash

Integrity Logs	`POST /api/integrity-logs` `GET /api/submissions/:submissionId/integrity-logs`	Submission id, event type, event details	Log objects

Cases	`GET /api/cases`
	`POST /api/cases`
	`PATCH /api/cases/:id`
		Submission id, verdict, status update	Case objects

1. Auth Module

APIs

- `POST /api/auth/signup`
- `POST /api/auth/login`
- `GET /api/auth/me`

Input

Endpoint	Input	Notes
`POST /api/auth/signup`	`email`, `password`, optional `role`	Only `student` is allowed through public signup.
`POST /api/auth/login`	`email`, `password`	Requires a confirmed Supabase auth account if email confirmation is enabled.
`GET /api/auth/me`	`Authorization: Bearer <token>`	Token is validated using Supabase Auth.

Output

- `signup` returns `user`, `session`, and `profile`
- `login` returns `user`, `session`, and `profile`
- `me` returns `id`, `authUserId`, `email`, `role`

Example

```
{
  "request": {
    "method": "POST",
    "url": "/api/auth/login",
    "body": {
      "email": "group8postmanstudent@gmail.com",
      "password": "Student@123"
    }
  },
  "response": {
    "user": { "id": "supabase-auth-uuid", "email": "group8postmanstudent@gmail.com" },
    "session": { "access_token": "jwt-token" },
    "profile": { "id": 6, "email": "group8postmanstudent@gmail.com", "role": "student" }
  }
}
```

2. Users Module

APIs

- `GET /api/users`
- `POST /api/users`

Input

Endpoint	Input	Notes
`GET /api/users`	Bearer token	Allowed roles: `admin`, `faculty`, `proctor`.
`POST /api/users`	`email`, `role`, optional `password_hash`, optional `auth_user_id`	Admin-only route.

Output

- `GET` returns an array of user rows.
- `POST` returns the created user row.

Example

```
{
  "request": {
    "method": "POST",
    "url": "/api/users",
    "body": {
      "email": "faculty1@college.com",
      "role": "faculty",
      "auth_user_id": "auth-uuid"
    }
  },
  "response": {
    "id": 8,
    "email": "faculty1@college.com",
    "role": "faculty",
    "auth_user_id": "auth-uuid"
  }
}
```

3. Exams Module

APIs

- `GET /api/exams`
- `GET /api/exams/:id`
- `POST /api/exams`
- `POST /api/exams/:id/questions`

Input

Endpoint	Input	Notes
`GET /api/exams`	Bearer token	Any authenticated user can read exams.
`GET /api/exams/:id`	Bearer token, exam id path param	Returns one exam with questions.
`POST /api/exams`	`title`, `start_time`, `end_time`, optional `config_json`	Allowed roles: `admin`, `faculty`.
`POST /api/exams/:id/questions`	`text`, `type`, optional `correct_answer`, optional `marks`	Exam must exist before question creation.

Output

- Exam list returns exam objects with nested questions.
- Create exam returns the inserted exam row.
- Create question returns the inserted question row.

Example

```
{
  "request": {
    "method": "POST",
    "url": "/api/exams",
    "body": {
      "title": "Postman Active Exam",
      "start_time": "2026-03-27T10:00:00.000Z",
      "end_time": "2026-03-27T12:00:00.000Z",
      "config_json": {
        "duration_minutes": 60,
        "tab_switch_limit": 3
      }
    }
  },
  "response": {
```

```
    "id": 2,
    "title": "Postman Active Exam",
    "start_time": "2026-03-27T10:00:00.000Z",
    "end_time": "2026-03-27T12:00:00.000Z",
    "created_by": 7
  }
}
```

4. Submissions Module

APIs

- `POST /api/submissions/start`
- `POST /api/submissions/:id/save`
- `POST /api/submissions/:id/submit`
- `GET /api/submissions/:id`

Input

Endpoint	Input	Notes
`POST /api/submissions/start`	`exam_id`, optional `answer_data`	Student-only. Exam must be active.
`POST /api/submissions/:id/save`	`answer_data`	Student can only save their own draft submission.
`POST /api/submissions/:id/submit`	`answer_data`	Student can only submit their own active submission.
`GET /api/submissions/:id`	Submission id path param	Students can view only their own; staff can view wider data.

Output

- Start returns the submission row, usually with `status: "draft"`.
- Save returns the updated draft submission.
- Submit returns the updated submission with `final_hash` and `status: "submitted"`.
- Get returns the submission plus related exam and user details.

Example

```
{
  "request": {
    "method": "POST",
    "url": "/api/submissions/start",
    "body": {
      "exam_id": 2,
```

```
      "answer_data": [
        {
          "question_id": 4,
          "answer": "Database Management System"
        }
      ]
    }
  },
  "response": {
    "id": 2,
    "student_id": 6,
    "exam_id": 2,
    "answer_data": [
      {
        "question_id": 4,
        "answer": "Database Management System"
      }
    ],
    "status": "draft"
  }
}
```

5. Integrity Logs Module

APIs

- `POST /api/integrity-logs`
- `GET /api/submissions/:submissionId/integrity-logs`

Input

Endpoint	Input	Notes
`POST /api/integrity-logs`	`submission_id`, `event_type`, optional `event_details`	Allowed roles: `student`, `proctor`, `admin`.
`GET /api/submissions/:submissionId/integrity-logs`	Submission id path param	Allowed roles: `admin`, `faculty`, `proctor`.

Output

- Create returns the inserted integrity log row.
- Get returns a list of log rows ordered by timestamp descending.

Example

```
{
  "request": {
    "method": "POST",
    "url": "/api/integrity-logs",
    "body": {
      "submission_id": 2,
      "event_type": "tab_switch",
      "event_details": {
        "count": 1,
        "source": "postman"
      }
    }
  },
  "response": {
    "id": 3,
    "submission_id": 2,
    "event_type": "tab_switch",
    "event_details": {
      "count": 1,
      "source": "postman"
    }
  }
}
```

6. Cases Module

APIs

- `GET /api/cases`
- `POST /api/cases`
- `PATCH /api/cases/:id`

Input

Endpoint	Input	Notes
`GET /api/cases`	Bearer token	Allowed roles: `admin`, `proctor`.
`POST /api/cases`	`submission_id`, optional `verdict`	Submission must exist. Existing case is returned if already present.
`PATCH /api/cases/:id`	One or more of `status`, `verdict`, `resolved_at`	If status becomes `resolved`, backend auto-fills `resolved_at`.

Output

- List returns case rows with related submission and proctor email data.
- Create returns the created or existing case.
- Patch returns the updated case row.

Example

```
{
  "request": {
    "method": "POST",
    "url": "/api/cases",
    "body": {
      "submission_id": 2,
      "verdict": "Open for manual review"
    }
  },
  "response": {
    "id": 2,
    "submission_id": 2,
    "proctor_id": 7,
    "status": "open",
    "verdict": "Open for manual review"
  }
}
```

Common Protected Input

Most protected endpoints require the same authentication input:

```
{
  "headers": {
    "Authorization": "Bearer <access_token>"
  }
}
```

Common Error Output Format

The controllers use shared helpers from `utils/http.js`, so errors usually follow this shape:

```
{
  "error": "Forbidden",
  "message": "This exam is not currently active."
}
```

Final Notes

- The backend currently uses JSON request bodies for all write operations.
- The backend returns JSON responses for both success and failure.
- Role checks are backend-enforced, not trusted from client input.
- Students are intentionally restricted to their own submissions and logs.